

2026 电赛 H 题 | 视觉闭环球平衡与运动控制系统

Vision-Based Ball Balancing & Motion Control System
2026 电子设计竞赛 H 题参赛项目

本项目面向 2026 年电子设计竞赛 H 题，实现了一套基于 **机器视觉 + STM32 + PID 闭环控制** 的球位置稳定与运动控制系统。

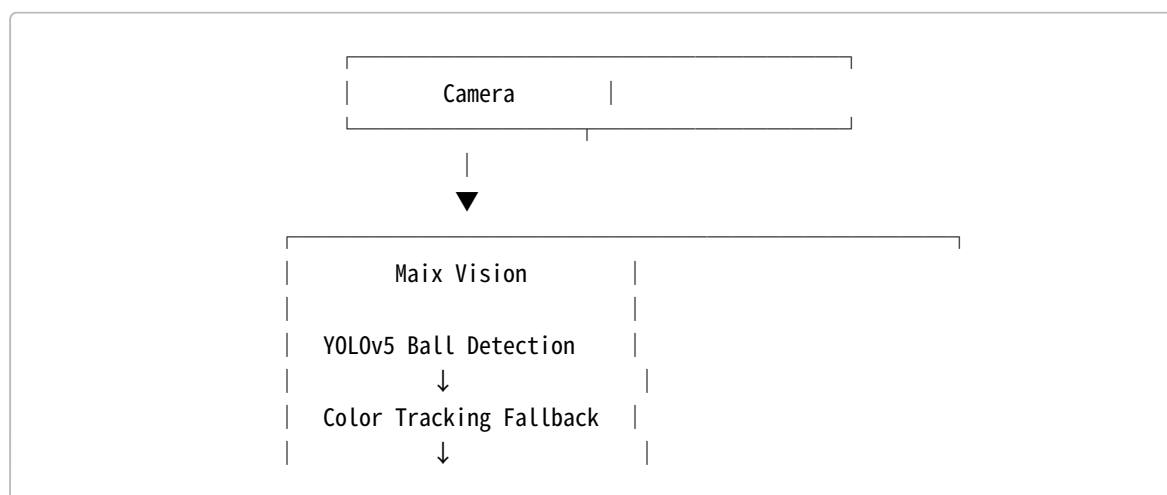
视觉端实时识别小球位置，并通过串口向 STM32 发送位置偏差及运动信息；STM32 根据视觉反馈运行 PID 控制算法，驱动执行机构调整球的位置。

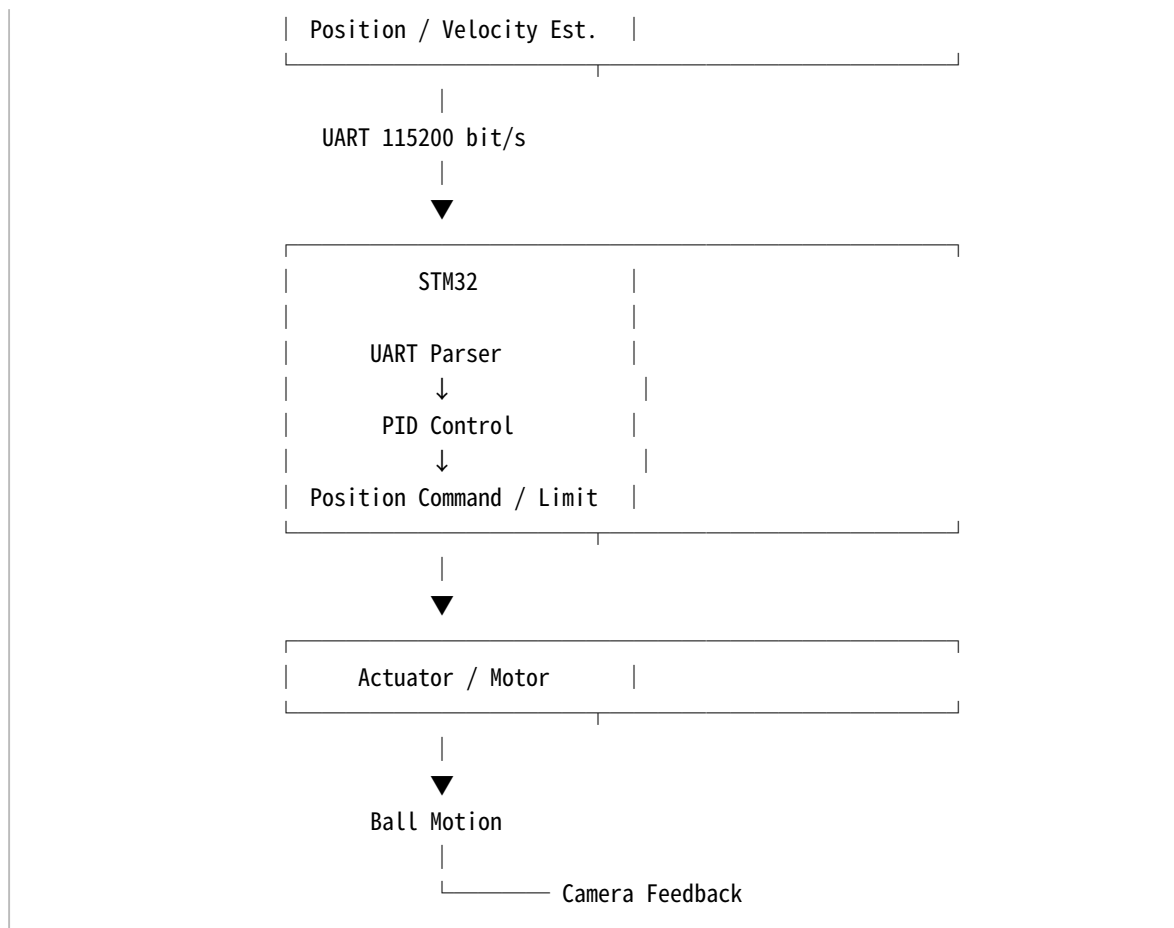
在赛前实际调试中，系统能够将小球稳定控制在目标中心附近，**稳态位置误差约为 2 cm**。

✨ 项目特点

- 基于 YOLOv5 实现小球目标检测
- 固定 ROI 区域检测，降低视觉计算量
- YOLO + 颜色跟踪双重目标跟踪策略
- 圆度、面积、宽高比多重目标筛选
- 目标短暂丢失时进行运动趋势预测
- 实时计算小球位置偏差 dx / dy
- 实时估计目标运动速度 vx / vy
- UART 二进制协议实现视觉端与 STM32 通信
- STM32 PID 闭环控制
- PID 死区处理，降低中心附近高频震荡
- PID 输出限幅，避免执行机构动作过大
- 支持 VOFA+ 在线调整 PID 参数
- 支持预设运动序列 / 视觉闭环双模式
- WebRTC 实时查看视觉识别画面

🧠 系统架构





整个系统形成：

视觉感知 → 状态估计 → 串口通信 → PID 控制 → 执行机构 → 视觉反馈

的完整闭环。

视觉识别

视觉部分基于 Maix 平台开发，使用 YOLOv5 网络检测小球。

主要流程：

```

graph TD
    A[Camera] --> B[固定 ROI]
    B --> C[YOLOv5 检测]
    C --> D[面积 / 宽高比筛选]
    D --> E[选择置信度最高目标]
  
```

The main process for visual recognition is as follows: Camera input is processed through a fixed ROI, then YOLOv5 detection, followed by area/aspect ratio filtering, and finally selecting the target with the highest confidence.

```

↓
颜色特征更新
↓
计算球心坐标
↓
计算 dx / dy
↓
估计 vx / vy
↓
UART → STM32

```

固定 ROI

为了减少无关区域对检测结果的影响，并降低推理计算量，程序只在小球主要运动区域内进行检测。

```

ROI_W = 340
ROI_H = 50

```

同时支持自动将 ROI 放置在图像中心区域。

YOLO + 颜色跟踪

仅依赖 YOLO 时，当小球出现：

- 快速运动
- 运动模糊
- 短暂遮挡
- 置信度下降

可能发生瞬间丢失。

因此本项目采用：

```

YOLO
|
├── 检测成功
│   ↓
│   更新目标颜色
└── 检测失败
    ↓
    Color Tracking
    ↓
    Roundness Filter

```

↓
Ball Position

YOLO 检测成功后提取目标附近的 LAB 颜色信息。

当 YOLO 短暂无法识别时，使用颜色阈值寻找候选 Blob，并通过圆度进一步过滤：

$$\text{Roundness} = 4\pi A / P^2$$

其中：

- A：Blob 面积
- P：Blob 周长

越接近圆形，该值越接近 1。



运动预测

程序同时记录目标中心位置变化：

$$\begin{aligned}\Delta x &= x(k) - x(k-1) \\ \Delta y &= y(k) - y(k-1)\end{aligned}$$

并进行指数平滑：

$$\begin{aligned}v_x &= \text{ALPHA_VX} * \text{delta_x} + (1 - \text{ALPHA_VX}) * v_x \\ v_y &= \text{ALPHA_VY} * \text{delta_y} + (1 - \text{ALPHA_VY}) * v_y\end{aligned}$$

当视觉目标短暂丢失时，根据当前速度估计进行有限帧数的位置预测：

$$\begin{aligned}x(k+1) &= x(k) + v_x \\ y(k+1) &= y(k) + v_y\end{aligned}$$

从而减少检测结果偶尔跳变或短暂丢失对控制系统造成的影响。

这里采用的是基于速度估计的短时运动预测，而不是完整的卡尔曼滤波器。



UART 通信

视觉端与 STM32 之间使用 UART 通信：

Baud Rate: 115200

视觉端发送：

\$ + dx + dy + vx + vy + state + #

数据帧：

字段	类型	说明
\$	uint8	帧头
dx	int16	X 方向位置偏差
dy	int16	Y 方向位置偏差
vx	int16	X 方向速度估计
vy	int16	Y 方向速度估计
state	uint8	是否检测到目标
#	uint8	帧尾

当目标丢失时：

```
dx = 0x7FFF
dy = 0x7FFF
vx = 0
vy = 0
state = 0
```

STM32 接收到有效目标数据后进入闭环控制。

STM32 控制

STM32 端主要负责：

```
Camera Data
↓
UART Parser
↓
Position Error
↓
PID Controller
↓
Output Limit
```

↓
Motor Target Position
↓
Motor

控制器采用位置式 PID：

$$u(k) = K_p \cdot e(k) + K_i \cdot \sum e(k) + K_d \cdot [e(k) - e(k-1)]$$

工程中支持：

Kp
Ki
Kd

实时调整。

PID 死区

当小球已经非常接近中心位置时，如果仍然持续进行高增益 PID 调节，很容易出现：

左 → 右 → 左 → 右

的高频震荡。

因此程序加入中心死区：

```
#define M2_DEAD_ZONE 5
```

误差进入死区后降低控制动作，从而提高中心位置稳定性。

输出限幅

为了避免瞬间误差过大导致执行机构动作幅度过大，对 PID 输出以及电机位置进行限制。

```
#define M2_PID_MAX_OUT 20.0f  
#define M2_MOTOR_MAX_DEV 300
```

从而提高系统运行安全性和稳定性。

VOFA+ 在线调参

STM32 端预留 VOFA+ 调参接口，可以在运行过程中修改：

Kp
Ki
Kd

无需每次：

修改代码
→ 编译
→ 下载
→ 测试

可以直接观察波形并调整参数，提高 PID 调试效率。

双模式控制

项目中保留了两种控制模式。

Mode 1：预设运动序列

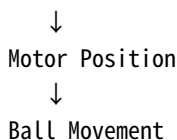
按照提前设定的位置、速度和保持时间运行：

Position 1
↓
Position 2
↓
Position 3
↓
...

主要用于机构测试以及比赛流程动作执行。

Mode 2：视觉 PID 闭环

Camera
↓
Ball Position
↓
PID



根据小球实时位置自动调整执行机构。



实际测试效果

在赛前完成的系统联调中：

项目	表现
小球检测	可稳定识别
视觉刷新	满足闭环控制需求
串口通信	稳定
PID 控制	可稳定收敛
中心稳定误差	约 2 cm
短时目标丢失	支持跟踪 / 预测
在线 PID 调参	支持
WebRTC 监控	支持

在正常测试环境下，小球能够快速回到目标区域，并稳定保持在中心附近。

≈ 2 cm 为项目调试阶段的实际测试结果，并非官方比赛测量结果。



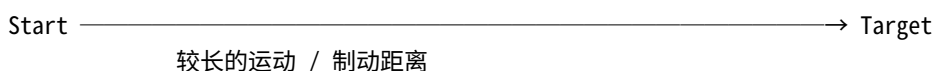
比赛结果与赛场复盘

本项目最终**未获得比赛奖项**。

主要问题并不是球平衡闭环本身，而出现在整车最终的**减速与停车阶段**。

赛前学校提供的训练地图与正式比赛地图在起跑区域长度上存在明显差异：

学校训练地图：



正式比赛地图：

Start —————> Target
较短的运动 / 制动距离

赛前车辆的：

- 行驶速度
- 动作切换时间
- 减速位置
- 制动距离

均基于训练地图进行了大量标定。

正式比赛时起跑段明显缩短，使原有控制参数与制动策略无法适应新的场地尺寸，车辆进入停车阶段时剩余制动距离不足，最终未能在规定位置内完成停车。

同校参加 H 题的队伍在正式比赛中均出现了类似的停车问题。

因此本项目最终未能获得奖项。

从比赛中发现的问题

这次比赛也暴露出了系统设计中一个非常重要的问题：

控制系统不应该过度依赖固定场地尺寸和固定时间参数。

原方案一定程度上采用了：

运行固定距离 / 时间
↓
执行预定动作
↓
固定位置开始制动

这种方案在场地完全一致时效果很好，但一旦：

地图尺寸改变
速度变化
摩擦系数改变
电池电压改变

控制效果就可能明显下降。

后续改进方向

如果重新设计比赛版本，将重点修改停车策略。

1. 使用视觉 / 标志物触发停车

避免：

运行 X 秒后停车

改为：

检测停车线
↓
测量距离
↓
进入减速状态
↓
动态制动

使控制系统不依赖固定地图长度。

2. 加入完整状态机

例如：

STATE_START
↓
STATE_ACCELERATE
↓
STATE_CRUISE
↓
STATE_APPROACH
↓
STATE_BRAKE
↓
STATE_STOP

每个状态由传感器条件切换，而不是单纯依赖 Delay。

3. 根据速度动态计算制动距离

利用：

v
a

动态估算停车距离：

$$d = v^2 / 2a$$

速度越高，越提前减速。

相比写死停车位置，可以显著提高不同场地条件下的适应性。

4. 增强视觉跟踪算法

当前已经实现：

YOLO
+
Color Tracking
+
Velocity Prediction

后续可以进一步加入：

Kalman Filter

建立：

[x, y, vx, vy]

状态模型，提高高速运动情况下的位置估计稳定性。

5. 减少硬编码参数

比赛系统中应尽可能将：

地图长度
停车位置
速度
PID 参数
动作时间

改为可在线配置参数。

这样即使比赛现场与训练环境存在差异，也能够快速重新标定。

主要代码

STM32 工程中的主要模块包括：

```
USER/
├── MiniBalance.c      # 主控制逻辑 / 双模式控制
├── Pid.c              # PID 算法
└── Pid.h

SYSTEM/
├── uart/
│   ├── uart.c        # 相机、电机、VOFA+ 串口通信
│   └── uart.h
└── encoder/
    ├── encoder.c      # 编码器读取
    └── encoder.h

CONTROL/
├── control/
│   ├── control.c
│   └── control.h

Vision/
└── main.py           # YOLO + 颜色跟踪 + 运动预测
```

技术栈

Embedded

```
STM32F103
C
Keil MDK
STM32 Standard Peripheral Library
UART
PID Control
Motor Position Control
VOFA+
```

Vision

```
MaixPy
Python
YOLOv5
LAB Color Tracking
Blob Detection
Motion Prediction
WebRTC
```



建议添加演示视频

如果有比赛前成功的视频，建议上传到：

```
assets/demo.mp4
```

或者上传至 Bilibili / YouTube，然后在 README 中加入：

📺 Demo

赛前调试过程中，小球能够稳定保持在目标中心附近，
实际稳态误差约 2 cm。

[▶ 点击查看 Demo](你的视频链接)

强烈建议放一段成功运行视频。

对于 GitHub 项目而言，一段十几秒的实际运行 Demo 往往比大量文字更能体现项目完成度。



建议添加图片

建议仓库增加：

```
assets/
├── system.jpg
├── vision.jpg
├── vehicle.jpg
└── competition.jpg
```

然后在 README 中展示：

📷 Project Preview

Hardware

![Hardware](assets/system.jpg)

Vision Detection

![Vision](assets/vision.jpg)

Vehicle

![Vehicle](assets/vehicle.jpg)

项目总结

这次比赛最终没有获得奖项，但项目完整经历了：

机械结构搭建
↓
嵌入式驱动
↓
串口协议设计
↓
机器视觉识别
↓
PID 控制
↓
系统联调
↓
现场比赛
↓
问题复盘

赛前测试中，球平衡闭环已经能够达到约 **2 cm 的中心稳定误差**。

正式比赛中，由于训练场地与正式比赛场地在关键起跑 / 制动距离上存在差异，原本基于训练地图完成标定的停车策略适应性不足，最终导致停车失败。

这次经历也让项目从单纯的“参数调通”，进一步认识到工程控制系统中：

鲁棒性、状态机设计、传感器反馈以及对环境变化的适应能力，与单一测试环境下的控制精度同样重要。

Disclaimer

本项目为电子设计竞赛期间完成的学习与竞赛项目，代码主要用于技术交流、学习和比赛复盘。